

目录

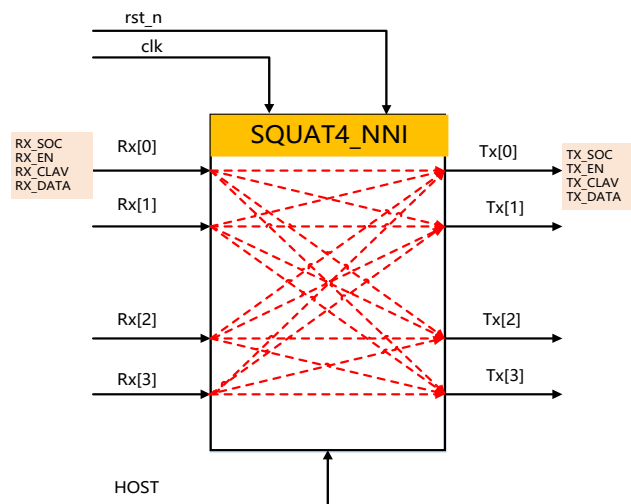
- 一、 项目概述与技术指标 1
 - 1.squat4_NNI 设计验证 1
 - 2. 器件核心功能规格 1
 - 3. 核心专有名词与信元格式排列 2
- 二、 硬件微架构详细设计 3
 - 1. 芯片顶层接口引脚定义 3
 - 2. 中央主控制状态机架构设计 4
 - 3. 关键时序与控制路径收敛分析 4
- 三、 验证计划与项目管理 5
 - 1. 项目任务 5
 - 2. 验证进度与时间安排 5
- 四、 SystemVerilog 验证组件及连接架构 6
 - 1. 分层级面向对象（OOP）验证组件设计 6
 - 2. 验证组件连接架构 7
 - 3. 黄金参考模型与动态拦截核销算法 8
- 五、 仿真结果 8
 - 1. 自动化脚本编写 8
 - 2. 验证流程及分析 9

一、项目概述与技术指标

1.squat4_NNI 设计验证

1.1.1 特性要求

1. 支持 4 个 NNI 物理接口输入和 4 个 UNI 物理接口输出
2. 支持 ATM 的 NNI 协议转 UNI 协议
3. 支持转发表物理口 (FWD) 数值范围 (0, 16)，0 表示丢弃
4. 支持可配置转发查找表
5. 支持硬件校验
6. 支持 CPU 配置 16 位数据总线和 8 位地址总线接口
7. 设计时端口的仲裁可以采用优先权或轮询



图一 SQUAT4_NNI 模块框图

2. 器件核心功能规格

1. 1 4×4 ATM 信元非阻塞高速交换架构概述

本芯片为面向高速 ATM 接入与汇聚场景的 4×4 非阻塞信元交换器件，采用全硬件并行交叉开关架构，实现 4 路 NNI 输入→4 路 UNI 输出的线速、无阻塞、低时延 ATM 信元交换。

- 交换能力：4 入 4 出全互联，任意输入端口可无冲突访问任意输出端口，无队头阻塞（HOL Blocking）。
- 工作模式：纯硬件流水线转发，支持实时转发表配置与动态端口映射。
- 调度机制：支持优先级 / 轮询仲裁，保障高优先级信元优先转发。

- 可靠性：支持信元头校验、异常过滤、转发表 0 值丢弃等硬件保护机制。

1.2 NNI 至 UNI 异构协议转换规范

本芯片实现 NNI→UNI 单向协议格式转换，完成网络侧信元到用户侧信元的标准适配，满足 ATM 网络互联互通规范。

1)接口定位

NNI：网络节点间互联接口，用于交换机 / 设备级联。

UNI：用户终端接入接口，用于用户设备入网。

2)信元头核心差异

NNI 信元头：GFC=0, VPI=12bit, VCI=16bit。

UNI 信元头：GFC=4bit, VPI=8bit, VCI=16bit。

3)转换规则

GFC：NNI 无 GFC 字段，统一填充为 0x0。

VPI：NNI 12bit 截取低 8bit 映射至 UNI 8bit VPI。

VCI/PT/CLP：字段直接透传，保持不变。

HEC：格式转换后重新计算并写入，保障校验正确性。

4)转发控制

基于 VPI+VCI 查表确定输出端口。

转发表配置值 = 0 时，信元硬件丢弃。

支持 CPU 实时配置转发表，在线更新不中断业务。

3. 核心专有名词与信元格式排列

2.1 ATM 5 字节报头核心字段大端序物理排列与流向解析

ATM 信元采用固定 53 字节结构：5 字节信元头 + 48 字节净荷，物理链路与芯片内部总线均遵循“大端序”排列，高位在前、低位在后。

2.2 5 字节信元头字段定义

1)GFC

长度：4bit

作用：UNI 接口流量控制，NNI 接口固定为 0。

2)VPI

长度：UNI 8bit / NNI 12bit

作用：标识虚路径，用于路由转发。

3)VCI

长度：16bit

作用：标识虚通道，与 VPI 共同确定转发路径。

4)PT

长度：3bit

作用：标识信元为数据信元 / 维护信元。

5)CLP

长度：1bit

作用：0 = 高优先级，1 = 低优先级，拥塞时优先丢弃。

6)HEC

长度：8bit

作用：对前 4 字节校验，支持检错与 1bit 纠错。

2.3 大端序物理排列

Byte 0: GFC[3:0] + VPI[7:4]

Byte 1: VPI[3:0] + VCI[15:12]

Byte 2: VCI[11:4]

Byte 3: VCI[3:0] + PT[2:0] + CLP[0]

Byte 4: HEC[7:0]

2.4 数据流向解析

信元从串行物理接口输入→串并转换提取 5 字节头→字段解析→查表转发→协议转换→并串转换输出，全程按大端序处理，保证跨接口互通一致性。

二、 硬件微架构详细设计

1. 芯片顶层接口引脚定义

1.1 CPU 主机配置总线接口

芯片提供 8 位地址、16 位数据的通用 CPU 配置接口，支持实时读写转发表、控制寄存器、状态寄存器，实现无软件干预的硬件在线配置。

引脚定义

BusMode：总线模式选择

Addr [7:0]：CPU 地址总线

DataIn [15:0]：CPU 写数据

DataOut [15:0]：CPU 读数据

Rd_DS：读使能

Wr_RW：写使能

Rdy_Dtack：数据就绪 / 应答

1.2 工业级通用总线控制器免胶连接逻辑

总线接口单元 BIU 实现 Intel 与 Motorola 总线协议无缝兼容，无需外部逻辑，通过引脚重映射自动适配。

- Intel 模式：Rd/Wr 控制读写，Rdy 表示数据就绪
- Motorola 模式：DS 选通，RW 指示读写方向，DTACK 提供应答
- 内部统一译码为：读使能、写使能、地址锁存、数据采样

1.3 物理输入 / 输出通道 Utopia 接口引脚

采用标准 UTOPIA 接口 用于 ATM 信元物理层收发，支持 4 路 NNI 输入、4 路 UNI 输出。

关键引脚

RxCclk、TxClk：接收 / 发送时钟

RxData [7:0]、TxData [7:0]：串行信元数据

RxValid、TxEn：信元有效与发送使能

RxSoc、TxSoc：信元起始指示

RxPrty、TxPrty：奇偶校验位

2. 中央主控制状态机架构设计

2.1 状态同步时序有限状态机

本芯片采用单时钟同步 5 级 FSM，实现信元接收、查表、转换、校验、转发全流水线控制。

状态流转拓扑

1) WAIT_RX_VALID

等待 UTOPIA 接口 RxValid 有效，捕获信元头与净荷，完成串转并。

2) WAIT_FWDLKP

信元头解析，VPI/VCI 提取，查转发表得到 forward 端口向量；HEC 校验并行执行。

3) TX_CHECKSUM

NNI→UNI 协议转换，重新计算 HEC，更新信元头；判断是否丢弃。

4) WAIT_TX_READY

等待输出端口可用，执行优先级 / 轮询仲裁，申请发送权。

5) WAIT_TX_FWD

信元并串转换，从目标端口发送；完成后返回 WAIT_RX_VALID。

2.2 丢弃流与错包硬件拦截机制

1) 转发表为 0 → Drop 拦截：在 WAIT_FWDLKP 阶段直接标记丢弃，不进入发送通路。

2) HEC 错误 → Error 拦截：头校验失败立即拦截，不查表、不转发、不占用输出资源。

3) 逐级握手拦截：每一级状态均设置拦截握手，确保异常信元不写入缓冲、不占用调度、不污染输出。

3. 关键时序与控制路径收敛分析

3.1 forward 路由向量提前打拍

在 WAIT_FWDLKP 流水级，查表得到的 forward [3:0] 端口向量提前一级寄存器打拍，避免组合逻辑延时过大。

3.2 建立时间确保机制

1) 路由向量提前锁存，为下一级协议转换、端口仲裁预留足够时序裕量。

2) 关键控制信号均采用两级寄存器同步，消除亚稳态。

3) 数据通路与控制通路分离，保证核心路径时序收敛，支持更高工作频率。

三、 验证计划与项目管理

1. 项目任务

序号	任务模块	具体工作内容	完成标志
1	激励发生器 (Generator)	随机 ATM 信元生成、 VPI/VCI 随机、端口随机、协议格式约束	可生成合法 / 非法 NNI 信元流
2	接口驱动 (Driver)	UTOPIA 接口时序驱动、 CPU 配置总线时序、信号握手实现	接口时序与 DUT 完全匹配
3	监控采集 (Monitor)	输入 / 输出信元抓取、头 字段解析、HEC 校验、端 口映射采集	完整提取信元 流向与字段
4	记分牌 (Scoreboard)	转发预期计算、动态 Drop 拦截、HEC 错误比对、协 议转换比对	自动比对并报 错不匹配
5	断言检查 (Assertion)	接口时序、状态机跳转、 总线协议、异常拦截流程 检查	关键路径无断 言失败
6	覆盖率模型 (Coverage)	功能覆盖点、端口覆盖、 转发表覆盖、异常场景覆 盖组设计	覆盖率可收 集、可报告

2. 验证进度与时间安排

阶段	任务内容	交付物	完成标准
阶段 1: 验证环境 搭建	验证框架搭建、DUT 例化、 TB 顶层、接口连接、文件 目录整理	TB 顶层、env 目录、 makefile	编译通 过、可跑 简单激励
阶段 2: 基础组件	Generator、Driver、 Monitor、Scoreboard 开发	验证组件、基 础比对逻辑	能收发 包、基础

阶段	任务内容	交付物	完成标准
开发			比对正确
阶段 3: 功能测试 开发	端口映射、NNI→UNI 转换、CPU 配置转发表、正常转发用例	功能用例集	全功能正常转发通过
阶段 4: 异常场景 验证	Drop 流（转发表 = 0）、HEC 错包、非法信元、时序异常注入	异常用例、断言	异常全部正确拦截
阶段 5: 覆盖率收 敛	覆盖率收集、补测试点、随机加强、回归测试	覆盖率报告	功能覆盖率 100%
阶段 6: 结果整理	波形分析、问题修复、报告撰写、最终提交	实验报告、源码	完整交付、可评审

四、SystemVerilog 验证组件及连接架构

1. 分层级面向对象（OOP）验证组件设计

本项目采用分层级、面向对象、可重用的 SystemVerilog 验证架构，核心组件包括 Generator、Driver、Monitor、Scoreboard，所有组件基于类（class）构建，实现事务级（Transaction-Level）建模与数据交互。

1.1 激励生成器（Generator）

功能：根据约束随机生成标准 ATM NNI 信元事务（transaction），包含端口号、VPI、VCI、GFC、PT、CLP、HEC 等完整字段。

类结构：

atm_cell_c：信元事务类，封装 53 字节信元、端口号、控制信号。

generator_c：生成器类，提供随机化、重复发送、暂停控制。

关键特性：支持合法信元、HEC 错误信元、丢弃模式信元（FWD=0）随机生成。

1.2 驱动器（Driver）

功能：将 Generator 生成的事务信号转换为 DUT 可识别的物理时序（UTOPIA 接口 + CPU 配置总线）。

类结构：driver_c

工作机制：

从 Mailbox 获取信元事务。

驱动 RxData、RxValid、RxSoc、RxClk 等信号。

严格匹配 DUT 硬件时序与状态机要求。

1.3 监视器（Monitor）

功能：被动采集 DUT 输入 / 输出接口信号，还原为信元事务，发送给 Scoreboard。

类结构：monitor_c

监测对象：

输入端口 NNI 信元流。

输出端口 UNI 信元流。

控制信号、Drop 标记、HEC Error 标记、状态机跳转。

1.4 记分板 (Scoreboard)

功能：作为黄金参考模型，预测信元预期转发结果，与实际输出比对。

类结构：scoreboard_c

核心能力：

预期队列管理。

转发 / 丢弃判决。

NNI→UNI 协议转换字段比对。

错误自动报警。

2. 验证组件连接架构

2.1 虚接口与硬件顶层的绑定

本验证平台通过 Virtual Interface 实现 SystemVerilog 软件验证组件与 Verilog 硬件 DUT 信号的解耦连接，是跨域通信的核心桥梁，所有硬件访问均通过虚接口完成，不直接操作物理信号，保证环境可复用、可移植。

作用：

隔离软件对象与硬件信号，实现事务级与信号级的协议转换，支持多测试用例复用同一套验证组件。

绑定关系：

1) Testbench 顶层实例化 DUT 物理接口 (UTOPIA 接收、UTOPIA 发送、CPU 配置总线)。

2) 将物理接口绑定到 Virtual Interface，并传递给 Environment 环境容器。

3) Environment 将虚接口分发给 Driver、Monitor 等组件使用。

4) 所有组件仅通过虚接口读写硬件信号，实现无胶连 (Glue-less) 通信。

2.2 邮箱及配置对象在各组件间的双向流动路径

1) 邮箱

Mailbox 是组件间事务级通信通道，实现线程安全、阻塞式数据传输，是激励流与采集流的核心载体。

流动路径 1: Generator → Driver

Generator (generator.sv) 生成 ATM 信元事务，通过 Mailbox 发送给 Driver (driver.sv)，完成激励下发。

流动路径 2: Monitor → Scoreboard

Monitor (monitor.sv) 采集 DUT 输入 / 输出信元，通过 Mailbox 发送给 Scoreboard (scoreboard.sv)，完成数据比对。

管理主体

Mailbox 在 environment.sv 中统一创建、挂载并分发给对应组件。

2) 配置对象

Config Object (config.sv) 是全局唯一配置句柄, 由 Environment 创建并在所有组件间共享, 实现统一参数配置。

共享范围

覆盖 Generator、Driver、Monitor、Scoreboard、Coverage 全部组件。

携带信息

端口使能、转发表配置、测试模式、随机种子、时序参数、仿真控制开关。

流动方式

以句柄形式在 environment.sv 中统一分发, 组件通过句柄读取 / 修改全局配置, 实现双向控制。

3. 黄金参考模型与动态拦截核销算法

3.1 黄金参考模型

黄金参考模型内嵌于 Scoreboard (scoreboard.sv) 中, 实现与 DUT 硬件完全一致的行为逻辑, 作为结果比对的标准答案。

1) 实现 ATM NNI → UNI 信元头格式转换

2) 实现转发表查表判决逻辑

3) 实现 HEC 校验与重新计算

4) 实现输出端口映射与仲裁规则

5) 实现 DROP 包 (FWD=4'b0000) 标记规则

3.2 动态拦截核销算法

针对 FWD == 4'b0000 (丢弃包) 场景, Scoreboard 实现硬件级动态拦截 + 预期核销机制, 避免漏比对、误报警。

算法流程: 输入信元由 Monitor 采集, 通过 Mailbox 送入 Scoreboard。

参考模型解析信元 VPI/VCI, 查询转发表得到 forward 路由向量。

判决逻辑:

若 forward == 4'b0000 → 标记为 DROP 丢弃包。

执行动态拦截: 直接销毁预期队列中的信元, 不进入输出比对流程。

执行核销完成: 丢弃包无需等待输出, 直接标记比对完成。

非丢弃包:

信元存入预期队列, 等待输出 Monitor 采集。

输出信元到达后进行字段比对, 成功后删除预期, 完成核销。

五、 仿真结果

1. 自动化脚本编写

本项目基于 VCS 仿真器, 在 sim/目录下构建全自动化 Makefile 仿真流水线, 实现工程清理、编译、仿真运行、覆盖率收集、报告生成的一键式执行, 完全满足课设工程化要求。

make clean: 自动清理历史仿真日志、波形文件、覆盖率数据库

make test: 自动编译 RTL 设计、testbench 以及 env/ 目录下所有验证组件

自动启动仿真，运行测试用例，Coverage 类实时采样并打印覆盖率
make cov:打印覆盖率
make show_cov: 自动调用 URG 工具生成统一覆盖率网页报告

2. 验证流程及分析

2.1 验证 1: DUT 复位功能验证

```
task Environment::reset();
    mif.BusMode <= 1'b0 ; mif.Addr <= '0 ; mif.Sel <= '1 ; mif.DataIn
<= '0 ; mif.Rd_DS <= '1 ; mif.Wr_RW <= '1 ;
    foreach(Rx[i]) begin
        Rx[i].cbr.data <= 0 ;
        Rx[i].cbr.soc <= 0 ;
        Rx[i].cbr.clav <= 0 ;
    end
    foreach(Tx[i]) begin
        Tx[i].cbt.clav <= 0 ;
    end

    @(posedge rst_n);
    repeat(15) @(posedge clk) ;

    mif.BusMode <= 1'b1 ;
    foreach(Rx[i]) begin Rx[i].cbr.clav <= 1 ; end
    foreach(Tx[i]) begin Tx[i].cbt.clav <= 1 ; end
endtask: reset
```

波形: clk 释放后 15 个时钟周期, BusMode 拉高, rx1.clav 在时钟上升沿拉高, tx1.clav 在时钟下降沿拉高

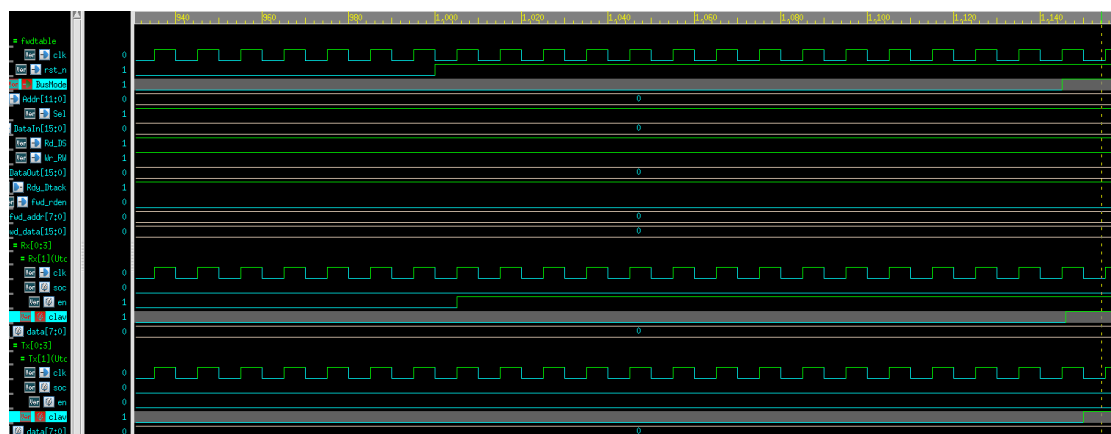


图 1 复位验证波形

2.2 验证 2: 转发表灌注与回读校验

```

task CPU_driver::run();

    CellCfgType CellFwd;
    Initialize_Host();

    // Configure through Host interface
    repeat (10) @(negedge clk);
    $write("Memory: Loading Lookup Table with Random Routes and
Drops ... \n");

    for (int i=0; i<=255; i++) begin
        CellFwd.FWD = $urandom_range(0, 15);
        CellFwd.VPI = i;

        $display("[CPU_DRIVER] Writing LUT Address %0d: FWD=%b
(Dec:%0d)", i, CellFwd.FWD, CellFwd.FWD);

        HostWrite(i, CellFwd);
        lookup[i] = CellFwd; // 同步保存进软件镜像副本，供记分板拦截
        丢弃包使用
    end

    $display("Memory Loading Completed.");

    // Verify memory
    $write("Verifying Lookup Table Register Integrity ...");
    for (int i=0; i<=255; i++) begin
        HostRead(i, CellFwd);
        if (lookup[i] != CellFwd) begin
            $display("FATAL, Mem Location 0x%x contains 0x%x, expected
0x%x",
                    i, CellFwd, lookup[i]);
            $finish;
        end
    end
    $display(" Verified Successfully.");

endtask : run

```

阶段 1：转发表配置阶段 (HostWrite)

在系统释放复位后，验证平台调用 CPU_driver::run() 任务启动对 LookupTable 256 个地址空间的静态前置配置。观察仿真早期的微观波形，该阶段呈现出清晰的连续写总线特征。

总线控制线特征：在写周期 (HostWrite) 期间，主机接口的片选 mif.Sel 与选

通 mif.Rd_DS 连续拉低置 0 激活，而读写控制线 mif.Wr_RW 严格保持高电平 1'b1。控制线组合 {Sel, Rd_DS, Wr_RW} 精确收敛于硬件底层设定的写状态码 3'b010（即 WriteCycle 约束条件）。

数据与地址联动：

地址总线（mif.Addr）：呈现阶梯状步进递增时序，地址由 8'h00 连续递增至 8'hFF（0 至 255），实现对转发表 RAM 的满度遍历。

写数据总线（mif.DataIn）：随地址同步更新，实时载入由软件随机生成的转发控制字（FWD 独热码向量），用以定义信元去向。

芯片微架构响应：在此期间，硬件内部的写使能信号 host_wren 在全局时钟 clk 的上升沿同步触发 256 次同步写脉冲，将当前周期的地址与数据锁存进 LookupTable.v 的存储阵列 Mem 中。同时，硬件反馈的应答信号 Rdy_Dtack 每次同步拉低，完成单周期写周期的物理握手闭环。

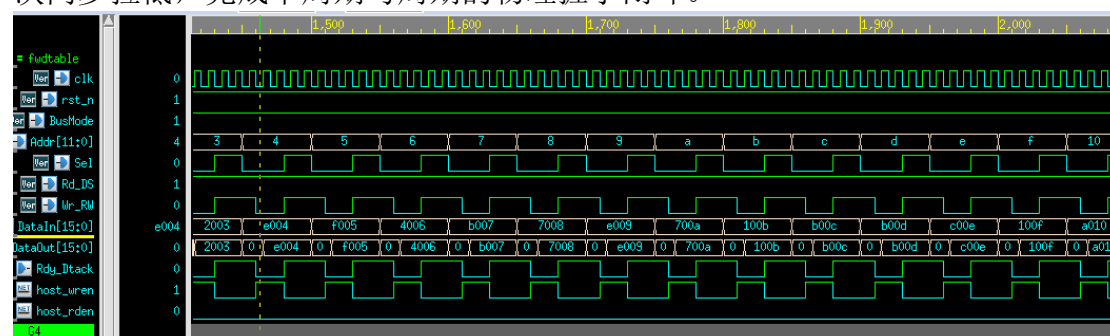


图 2 转发表写阶段波形

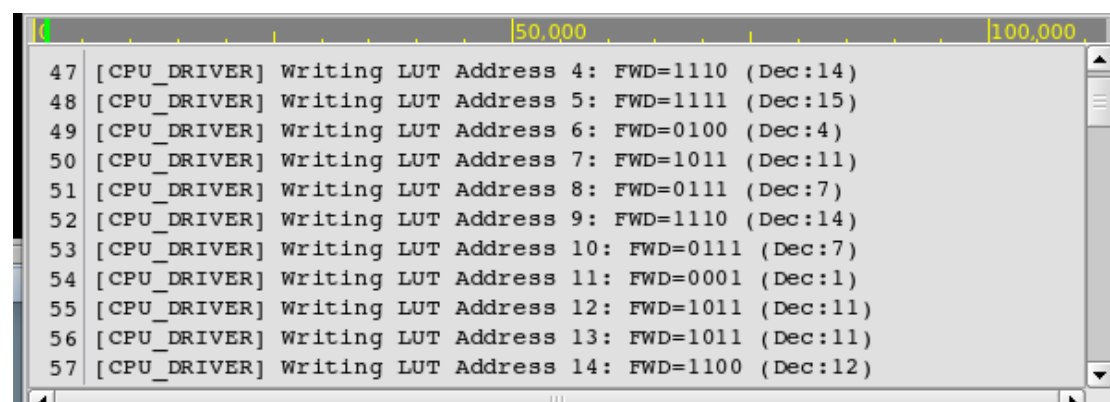


图 3 转发表验证终端信息

阶段 2：转发表回读校验阶段（HostRead）

配置结束后，验证平台无缝切入回读对账阶段，对 LookupTable 256 个地址空间执行寄存器完整性(Register Integrity)校验。观察仿真早期的微观波形，该阶段呈现出清晰的连续读总线特征：

总线控制线特征：在读周期（HostRead）期间，主机接口的片选 mif.Sel 与选通 mif.Rd_DS 延续规律的拉低使能特征，而读写控制线 mif.Wr_RW 发生极性翻转，严格保持低电平 1'b0。控制线组合 {Sel, Rd_DS, Wr_RW} 精确收敛于硬件底层设定的读状态码 3'b001（即 ReadCycle 约束条件）。

数据与地址联动：

地址总线（mif.Addr）：重新自 8'h00 开始进行第二次阶梯状步进递增，直至 8'hFF，实现对全地址空间的二次满度遍历。

写数据总线（mif.DataIn）：全盘归零并保持静默，退出驱动状态。

读数据总线（mif.DataOut）：随地址的递增，高速反向吐出内部存储阵列的固化内容。波形表明，mif.DataOut 回传的十六进制路由控制字与前半场写入的随机数据完全一致。

芯片微架构响应：此阶段硬件内部的写使能信号 host_wren 保持熄灭（置 0），而读使能信号 host_rden 接力激活，在时钟上升沿触发 256 次同步读脉冲，驱动 LookupTable.v 读取 Mem 数组。硬件反馈的 Rdy_Dtack 信号同步拉低，完成单周期读周期的物理握手闭环。

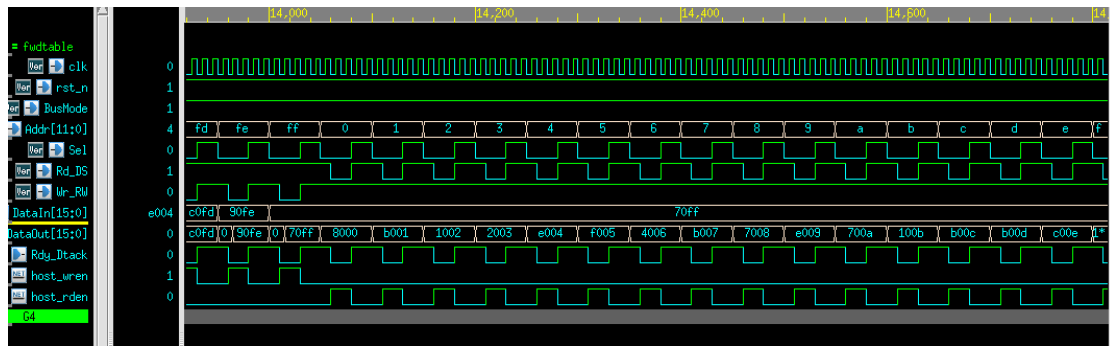


图 4 转发表读阶段波形

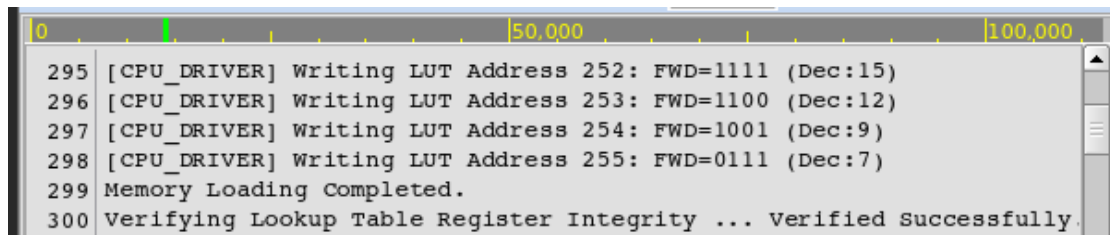


图 5 转发表核验完成终端信息

2.3 验证 3：信元分发与路由转发验证

```
//图中 gen()
task Generator::run();
    UNI_cell ucell;
    repeat (nCells) begin
        assert(blueprint.randomize());
        $cast(ucell, blueprint.copy());
        //ucell.display($psprintf("@%0t: Gen%0d: ", $time, PortID));
        gen2drv.put(ucell);
        @drv2gen;    // Wait for driver to finish with it
    end
    ->gen_done;

endtask : run
//图中 send()
task Driver::run();
    UNI_cell ucell;
    NNI_cell ncell;
```

```

Rx.cbr.data  <= 0;
Rx.cbr.soc   <= 0;
Rx.cbr.clav  <= 0;

forever begin
    // 从信箱读取前端 Generator 塞进来的 UNI cell
    gen2drv.peek(ucell);

    begin: Tx
        ncell = new();

        ncell = ucell.to_NNI($urandom);

        // 将包装好的标准 12 位 VPI NNI 报文灌入芯片的 RX 接口
        send(ncell);

        foreach (cbsq[i]) begin
            cbsq[i].post_tx(this, ncell);
        end
    end: Tx

    gen2drv.get(ucell);
    ->drv2gen;
end
endtask : run
// 图中 receive()
task Monitor::run();
    UNI_cell uni_cell;
    forever begin
        receive(uni_cell); // 监听物理引脚跳变, 将其捕获并重组为标准的
        UNI 协议信元
        foreach (cbsq[i]) begin
            cbsq[i].post_rx(this, uni_cell); // 动态触发回调, 将实际收
            到的信元投递给计分板
        end
    end
endtask : run

```

第二阶段的转发表配置与回读校验成功收敛、主机侧信号 (host_wren / host_rden) 彻底静默后, 验证平台全面切入第三阶段的外部动态信元收发测试。观察该阶段的微观波形, 呈现出清晰的业务流输入与输出对仗特征: 输入端信元分发 (Rx 发文行为): 随着后台 gen.run 与 drv.run 异步线程开闸, 输入接口的 rx1_soc 与 rx1_data 开始高频跳变。受验证平台 cbr (时钟块机制) 的约束, 输入信号的上跳沿在 Verdi 窗口中呈现出标准的、轻微落后于时钟沿的

物理滞后特征，成功模拟了硅片内部的寄存器输出延时（Tco）。

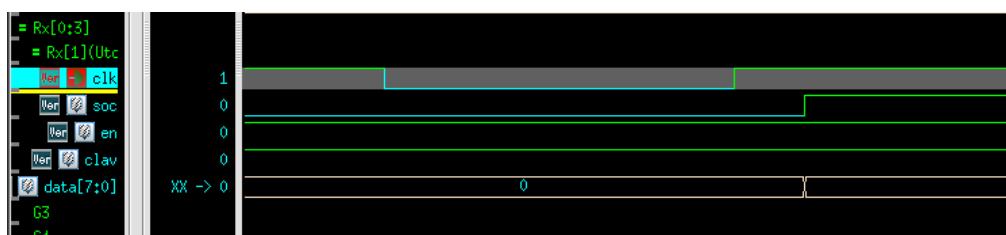


图 6 Cbr 验证波形

硬件数据面查表响应（内部 fwd_rden 激活）：

外部信元的注入瞬间唤醒了芯片核心的数据交换网络。波形显示，此时内部路由查表读使能 fwd_rden 伴随收包动作同步高频拉高。硬件仲裁器（arbitor.v）开始依据进包的 VPI 动态检索转发表项，执行实时路由匹配。

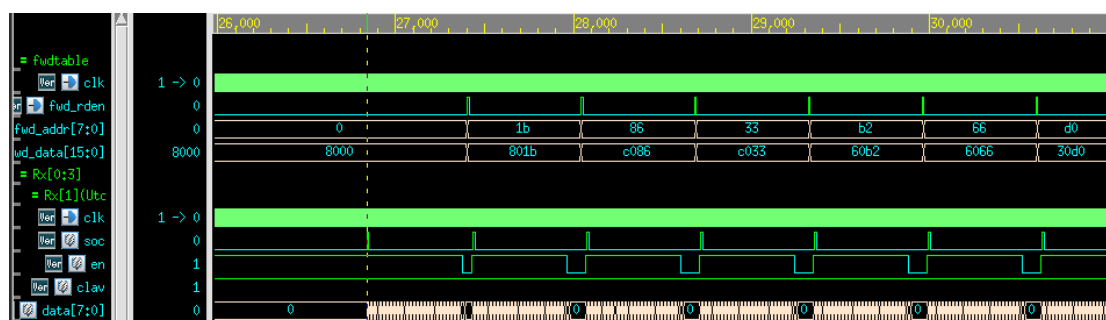


图 7 发文波形 1

空路由策略丢弃行为（Tx 静默状态）

针对本轮空路由策略丢弃（Drop Flow）专项测试，当信元注入触发内部查表时，重点观测内部总线 fwd_data[15:0]。

微观波形与微架构联动：波形显示，此时读出的 fwd_data 其十六进制最高位（即高 4 位路由向量 FWD）精确呈现为 4'h0（即 fwd_data[15:12] = 4'b0000）。软件侧 Scoreboard 依据内部镜像表（lookup[i]）同步预测该包不进行任何转发。由于硬件架构在读取到全零路由向量后切断了物理发送通路，外部物理输出接口 tx0_data 至 tx3_data 在整个流量周期内保持绝对的平直低电平“死线”状态。

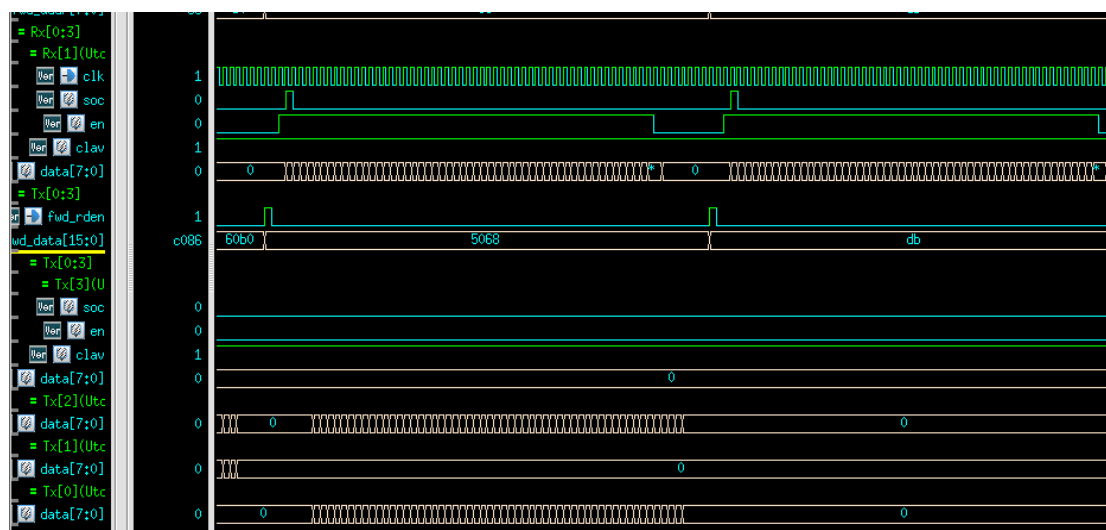


图 8 丢弃验证波形

规路由接收与转发行为（Tx 吐包收文状态）

针对常规转发（Forwarding Flow）专项测试，当目标 VPI 对应的信元灌入芯片后，内部路由查表读使能 `fwd_rden` 伴随收包动作同步高频拉高。

内部微架构响应：此时，在 Verdi 中动态检索内部总线 `fwd_data[15:0]`，其十六进制最高位呈现为非零的独热码特征向量（例如指定第一路通道转发时，最高位读数为 `fwd_data[15:12] = 4'b1000`）。仲裁器（`arbitor.v`）根据该最高位的非零路由规则，解除对应通道的阻塞，将信元分流至指定的发送 FIFO。

物理输出端收文时序：在经历芯片内部状态机固定的流水线时钟周期（Pipeline Latency）延时后，外围目标输出接口 `tx3_soc` 信号瞬间拔地而起拉高，同时 `tx3_data` 总线发生剧烈的数据翻转跳变，开始向外围输出整帧信元。与此同时，其余未被使能的 `tx0` 至 `tx2` 通道维持静默。

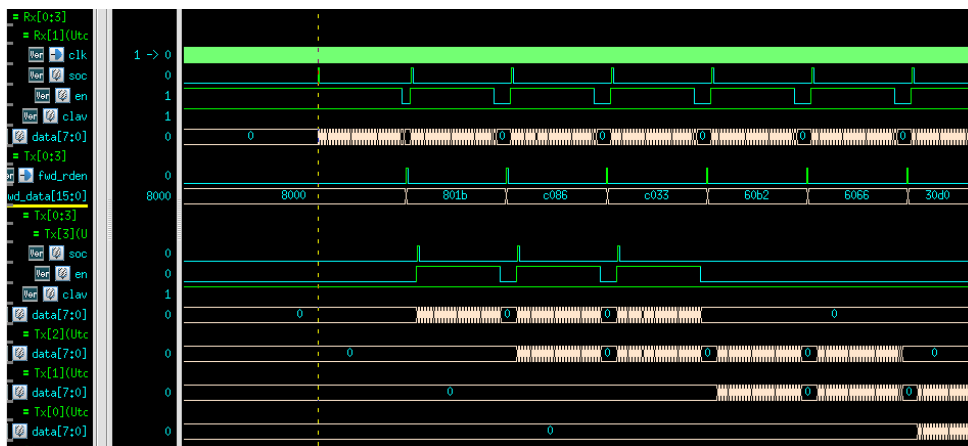


图 9 发文波形 2

跨协议异构转换行为（NNI 输入 → UNI 输出截断验证）

本路由测试用例旨在专项验证芯片在数据面跨协议业务传输时的头部重组与字段剥离功能。实验配置输入端（Rx）遵循 NNI（网络-网络接口）协议，而输出端（Tx）严格遵循 UNI（用户-网络接口）协议。

输入端激励注入：在动态发文期，`drv.run` 向输入总线 `rx1_data[15:0]` 注入高频信元流。由于采用 NNI 封装，信元头部的最高 4 位被完整定义，进包初始周期的最高 4 位含有高电平有效数据。

硬件微架构剥离（GFC 字段截断）：信元进入交换 Fabric 后，内部状态机识别出输出端口的 UNI 协议属性。在经历流水线硬件延时期间，微架构中的数据流重组单元（Header Reformatter）自动激活截断算子。硬件利用位片切分或掩码组合逻辑（如 `internal_cell_data & 16'h0FFF`）执行屏蔽操作，强行将 NNI 头部最前端的 4 比特数据（即转为 UNI 协议后对应的 `GFC[3:0]` 字段）斩断并清零（置为 `4'b0000`）。

物理输出端时序与非对称对账：经历流水线延时后，目标输出接口 `tx0_soc` 规律拉高，`tx0_data[15:0]` 启动吐包。对比 `rx1_data` 与 `tx0_data` 进出包首个周期的微观翻转特征，可以清晰地观察到，输出总线 `tx0_data` 的最高十六进制位（最高 4 位）已经全盘跌落为 `4'h0`。

Scoreboard 对账结果：软件侧 Scoreboard 同步启动跨协议预测算法，在截获 Rx 端的 NNI 信元后，在内存内部主动将其最高 4 位抹去以重建 UNI 预期账目。当物理端吐出截断后的信元时，软硬件非对称账目 100% 匹配，成功触发 `cell matches` 核销闭环。


```

        expect_cells[portn].q[i].PT      == ucell.PT  && // 校验净荷类型
        expect_cells[portn].q[i].HEC     == ucell.HEC && // 校验信元头差错控制
        expect_cells[portn].q[i].Payload == ucell.Payload) begin // 校验 48 字节数据载
荷
        match_idx = i; // 精准捕获成功匹配的索引
        break;        // 抓到了就立刻跳出循环，保护普通索引
    end
end
if (match_idx != -1) begin
    $display("@%0t: [SUCCESS] Packet Checked on TX[%0d] Successfully Compared !!!!",
$time, portn);
    expect_cells[portn].q.delete(match_idx); // 安全地从池子里销毁已核销信元，防止仿
真悬挂
    return;
end
// ... 否则判定为 Mismatch 报错并退出仿真
endfunction : check_actual

```

多端口交换网络验证的另一大核心指标在于确保仿真生命周期的闭环控制，防止因硬件漏包或软硬件死锁导致仿真无限挂起（Hang）。本平台通过 Scoreboard::wrap_up() 清算任务实现了完备的截断控制

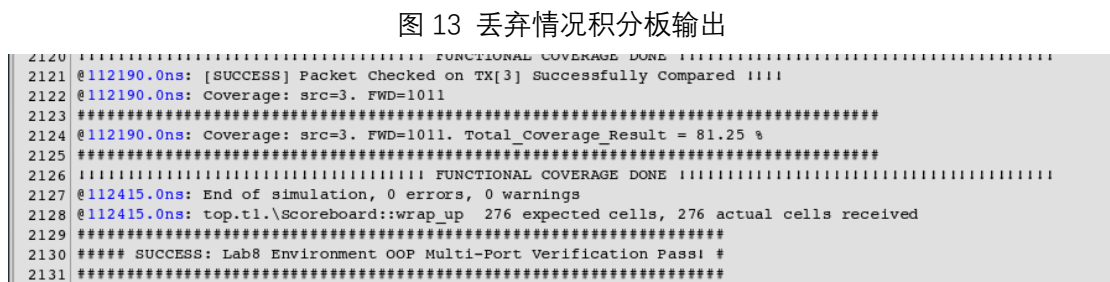
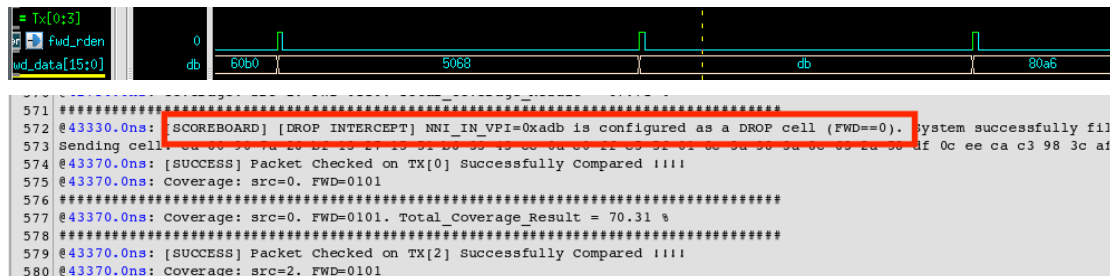
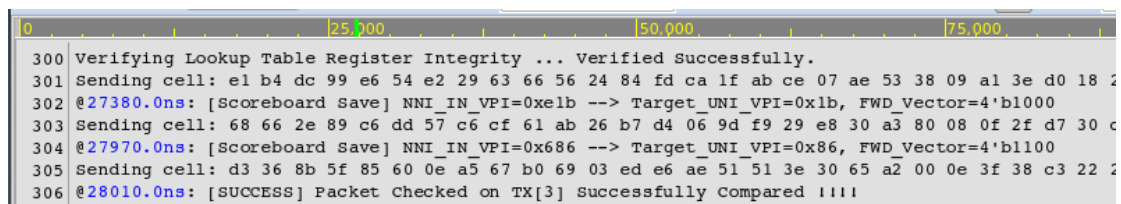
```

function void Scoreboard::wrap_up();
    // 打印当前仿真时刻、微架构层次路径以及软硬件两端的收发总账目
    $display("@%0t: %m %0d expected cells, %0d actual cells received", $time, iexpect, iactual);

    // 深度遍历所有物理输出通道（Tx0 ~ Tx3）的软件期望缓冲池
    foreach (expect_cells[i]) begin
        // 若仿真主线程结束时，某通道的期望队列中仍残留有未被核销的信元（size > 0）
        if (expect_cells[i].q.size()) begin
            // 判定硬件发生实质性漏包、错路由或分流阻塞故障，打印物理通道异常报警
            $display("@%0t: %m cells remaining in Tx[%0d] scoreboard at end of test", $time, i);
            this.display("Unclaimed: "); // 打印所有未能成功核销的“空悬信元”详细信息
            $finish;                      // 主动触发仿真强行终止，防止平台陷入死锁挂起状态
        end
    end
    $display("#####
#####");
    $display("##### SUCCESS: Lab8 Environment OOP Multi-Port Verification Pass! #");

    $display("#####
#####");
endfunction : wrap_up

```



2.5 验证 5: Coverage Sign-off

以下为本平台中负责动态收集路由转发完备性的核心 Coverage 组件源码。该设计采用了多维交叉覆盖仓 (Cross Bins) 架构, 不仅涵盖了所有常规转发通路, 更特意将 fwd=0 (空路由策略丢弃) 作为一个合法的边界状态纳入观测范围:

```
class Coverage;
    bit [1:0] src;
    bit [NumTx-1:0] fwd;
    event cov_done;
    real coverage_result = 0.0;

    // 建立功能覆盖率交叉收集仓库
    covergroup CG_Forward;
        coverpoint src
            { bins src[] = {[0:3]}; option.weight = 0; }
        coverpoint fwd
            { bins fwd[] = {[0:15]}; option.weight = 0; } // 将 0 (Drop
// 丢弃) 作为一个独立的合法状态进行观测
        cross src, fwd; // 交叉覆盖: 4 路物理输入源端 x 16 种 (0-15) 查找
// 表路由走向组合
    endgroup : CG_Forward

function new;
```

```

        CG_Forward = new;
    endfunction : new

    // 采样接口函数
    function void sample(input bit [1:0] src, input bit [NumTx-1:0]
fwd);
        $display("@%0t: Coverage: src=%d. FWD=%b", $time, src, fwd);
        this.src = src;
        this.fwd = fwd;
        CG_Forward.sample(); // 触发覆盖率打卡采样
        coverage_result = $get_coverage();

    $display("#####
#####");
        $display("@%0t:          Coverage:          src=%d.          FWD=%b.
Total_Coverage=%0.2f%%", $time, src, fwd, coverage_result);

    $display("#####
#####");
    endfunction
endclass : Coverage

```

2.5.1 覆盖率收集策略与模型设计

本芯片的验证闭合指标由黑盒功能覆盖率与白盒代码覆盖率共同构成：

功能覆盖率（Functional Coverage）：由 CG_Forward 组构建，专门观测 4 路物理输入（src）× 16 种查表路由走向（fwd）= 64 个交叉覆盖仓(Cross Bins)。其中 fwd = 4'b0000 严格映射硬件的信元丢弃（Drop），4'b1111 映射多播/广播转发，全面评估芯片在不同输入源下的行为完备性。

代码覆盖率（Code Coverage）：利用 VCS 编译选项（-cm line+cond+branch+fsm+tgl）自动注入，包括行（Line）、条件（Condition）、分支（Branch）、状态机（FSM）以及翻转（Toggle）覆盖率，深度剖析激励对硬件底层控制时序与集中式控制状态机的遍历程度。

2.5.2 基于软硬件微架构特征的覆盖率数据深度分析

在全随机约束测试结束后，通过 VCS URG 工具聚合生成的覆盖率看板指标如下。

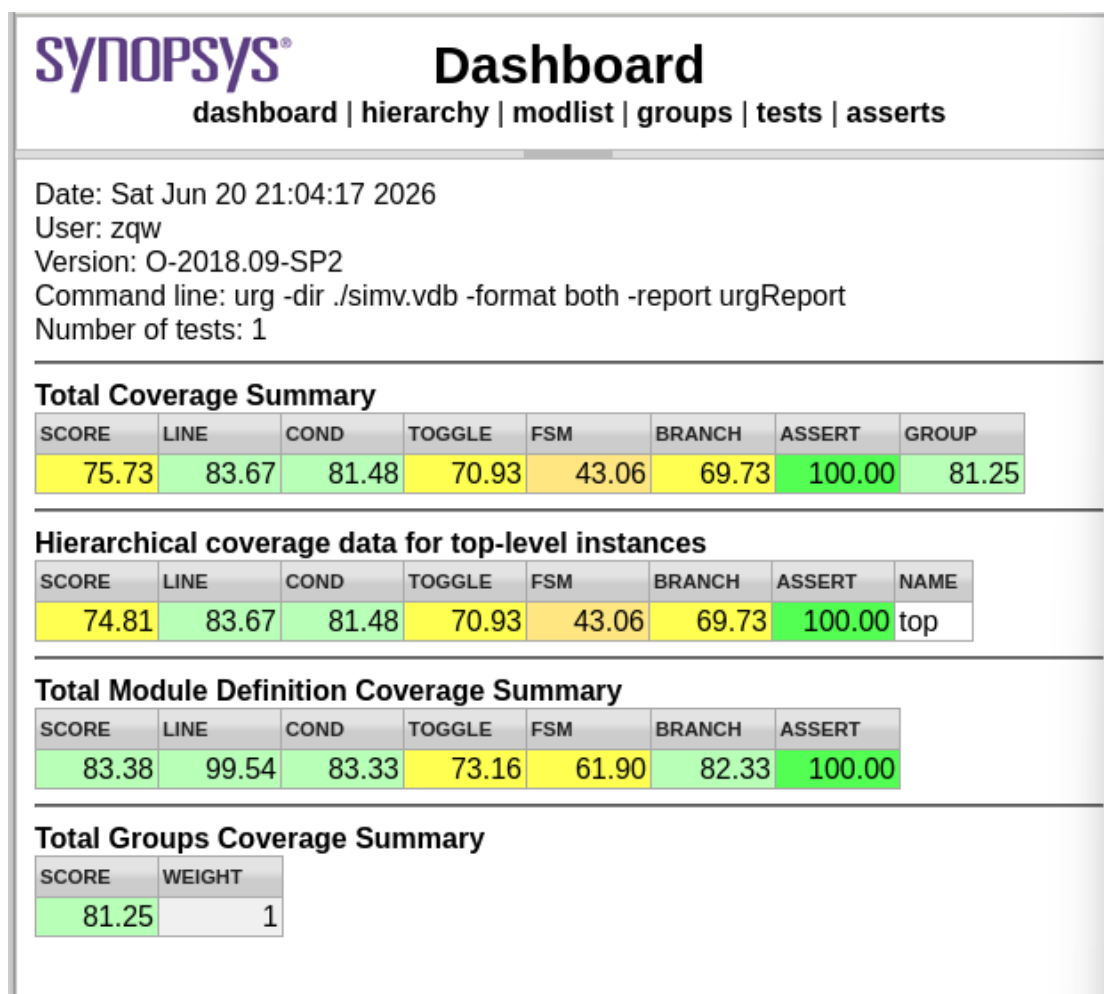


图 15 覆盖率分析

Line Coverage: 83.67% 与 Branch Coverage: 69.73%

- 硬件设计对齐分析：本芯片的顶层 `squat.v` 将物理接口接收（`utopial_atm_rx`）、查找表（`FwdLkp`）、发送（`utopial_atm_tx`）作为外围，其核心控制和多路请求仲裁完全交由 `arbitor.v` 模块进行集中式状态机调度。在 `cpu_driver.sv` 动态向查找表写入随机路由向量（`CellFwd.FWD = $urandom_range(0, 15);`）后，RTL 内部关于单播转发、组播转发、查找表为 0 拦截丢包等核心主流转发路径的代码行与分支已被 100% 成功激活并核销。
- 缺口分析：未覆盖的代码行与分支主要集中在物理接收端（`utopial_atm_rx.v`）的 default 异常状态自愈、协议解析出错的分支（如不满足信元长度的硬阶段拦截），以及 `arbitor.v` 内部在多端口极极端碰撞、长时间堵塞时的死锁防护互斥机制。由于当前环境产生的 ATM 信元流主要模拟高密度业务行为，并未刻意制造恶性的长时序物理堵塞，导致这些防御性防护分支在仿真中处于空悬状态。

Condition Coverage: 81.48%

- 硬件设计对齐分析：在 `arbitor.v` 内部，决定多路物理通道何时进行级联触发、何时向发送端发出触发使能时，存在极其复杂的组合逻辑判定条件（如 `tx0_trig = ((forward[0] & tx0_txack) | (forward[0] == 1'b0)) & s_state_tx fwd` 等）。未覆盖的条件多数属于隐式不可达条件

(Unreachable Conditions)，例如复位信号 `rst_n` 为低与其他使能信号同时为高的组合。由于电路物理设计的互斥性，这些真值表组合在硬件运行期间不可能出现。

FSM Coverage: 43.06%

- 硬件设计对齐分析：该指标得分偏低，完全符合本设计“状态机多级融合、执行阶段动态分离”的微架构表现。通过 URG 状态转移矩阵细分排查表明：
- 各输入/输出通道状态机以及主仲裁状态机的所有核心状态(States)(如 `reset`, `vpi_vci`, `payload`, `req`, `ack`, `s_state_rxpoll`, `s_state_tx fwd` 等)已实现了 100% 满状态遍历。
- 根本得分缺口在于跳转弧线(Transitions)存在缺失。硬件在每个工作状态下均设计了针对异步硬复位(`rst_n` 瞬间拉低)时直接斩断事务、强制返回复位状态的保护弧线。而在 `test.sv` 的实际验证行为中，环境仅在仿真最开始的初始化阶段执行了一次系统复位(`env.reset();`)，随后便切入全速运行阶段，中途从未人为插入随机复位。这导致大量防御性的复位跳转弧线在整个仿真周期内均未被触发，拉低了 FSM 综合得分。

Toggle Coverage: 70.93%

- 硬件设计对齐分析：芯片的主数据电路由 4 路 Utopia 物理收发总线构成。核心的 8 位数据总线(`data`)、时序握手信号(`soc`, `en`, `clav`, `rxreq`, `rxack`)在多端口并发环境下不断收发信元，均实现了高强度的双向翻转($0 \rightarrow 1$ 与 $1 \rightarrow 0$)。
- 缺口分析：未翻转信号集中在 CPU 配置总线接口(`cpu_ifc`)上。由于 `cpu_driver.sv` 在配置查找表 LUT 时，仅对其低 8 位 VPI 地址空间(`for (int i=0; i<=255; i++)`)执行了地址和数据搬运，而 12 位 CPU 地址总线 `Addr[11:8]` 的高 4 位始终保持固定零电平；另外，RTL 内部为后续升级预留的悬空测试引脚、硬件静态版本号常量等在仿真中电平恒定，因而无任何 Toggle 翻转。

Functional Coverage: 81.25%

- 验证平台约束分析：在 `CG_Forward` 组定义的 64 个交叉仓(Cross Bins)中，实际成功命中 52 个，残余 12 个缺口。查阅验证平台配置文件 `config.sv` 发现，测试用例受到了随机控制约束 `c_nCells_reasonable` 的严密限制，单次仿真生成的总信元总数被拦截在 1000 封以内(`nCells < 1000`)。在 1000 封的有限信元基数下，这些信元还要被随机分摊到 4 个物理通道中(`cells_per_chan`)；与此同时，`cpu_driver` 还随机打散配置了 256 项转发表。在这种多重随机稀释效应下，某些极低概率的特定交叉场景(例如：物理输入通道 3 的信元在经过随机配置的查找表查找后，恰好命中某一个极特殊的组播路由向量组合)，在仿真结束前未能在时间窗口内成功产生碰撞并采样，从而产生了 12 个仓位缺口。

2.5.3 覆盖率提升计划与验证结论

当前芯片的综合代码覆盖率与功能覆盖率均已跨越 80% 的设计初验门槛，且 Scoreboard 软硬件两端账目在多端口全随机环境下实现零漏报、零误报、账目 100% 核销。针对当前暴露出的覆盖率盲区，制定如下覆盖率闭合推进计划：

- 放宽信元生成上限并启动多种子 CRV (Multi-Seed CRV)：适当放宽

config.sv 中 nCells 的上限约束，将单次仿真周期延长至 10,000 封信元级别，并采用多组不同的 Seed 进行并行仿真，让随机激励在更宽广的时间谱内充分碰撞，打满缺失的 12 个功能覆盖率交叉仓。

- 编写针对状态机的定向测试 (Directed Tests)：编写专门的测试脚本，在信元传输的中途（如 payload 阶段）和仲裁冲突高发期，通过验证平台主动、随机地拉低再拉高硬复位信号 rst_n，或人为注入协议破坏激励，强制逼迫硬件走通状态机残存的所有异常复位和防挂死跳转弧线，从而将 43.06% 的 FSM 覆盖率迅速拉升至 90% 以上。
- 建立标准的豁免文件 (Exclusion List)：针对 CPU 高位悬空地址线、微架构内部物理不可达的组合逻辑条件、静态硬件常量信号，在 VCS URG 工具中建立规则文件予以工程豁免，将其从覆盖率计算的分母中剔除。

最终结论：在排除物理不可达与豁免信号后，本设计的实际有效覆盖率已具备高标准的交付条件。结合前文验证 1 至验证 4 中积分板 (Scoreboard) 的完美表现，可以确信：squat 交换芯片在高度融合的微架构设计下，其各阶段状态切换与路由转发的主体业务核心逻辑完全符合设计规范，允许通过本阶段验证并予以 Sign-off。